

AI Movie Recommendation System: Project Report

CodSoft Artificial Intelligence, Week 3 Project

1. Introduction

A recommendation system analyses information about items or users and suggests relevant content. Streaming platforms, online stores and news apps all rely on them. This project builds a **content-based movie recommender**: the user types the name of a movie they like, and the system returns the five most similar movies in its dataset.

Objective. Build an AI-powered system that recommends movies based on their similarity, while practising dataset handling, data preprocessing, feature selection, text processing, similarity calculation, recommendation logic and Python implementation.

Technology. Python, Pandas (data handling) and Scikit-learn (`CountVectorizer` , `cosine_similarity`).

2. Dataset

The project uses the **TMDB 5000 Movie Dataset** from Kaggle, which is distributed as two CSV files:

File	Contents
<code>tmdb_5000_movies.csv</code>	One row per movie (about 4,800 movies): id, title, overview, genres, keywords, budget, revenue, popularity, release date and more
<code>tmdb_5000_credits.csv</code>	One row per movie: movie_id, title, cast (full actor list) and crew (full crew list)

The genres, keywords, cast and crew columns are stored as **JSON-like strings**, for example `[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}]`. They cannot be used directly and must be parsed first.

3. Methodology

The system follows the eight-step pipeline from the project workflow: collect, explore, clean, select features, combine features, vectorize, calculate similarity, generate recommendations.

3.1 Loading and merging the data

The two files are merged into one table using the movie id (the credits file's `movie_id` column is renamed to `id`). The credits file's `title` column is dropped before merging, because the movies file already has one. Keeping both would create duplicate columns (`title_x`, `title_y`) and break later steps.

3.2 Exploring the data

Before changing anything, the dataset is inspected:

- **Shape and column names / data types** (`shape`, `info()`)
- **First rows** (`head()`) to see what the raw values look like
- **Missing values** per column (`isnull().sum()`)
- **Duplicate** rows and titles
- **Genre distribution**: genres are parsed and counted to see which are most common

This showed that a few columns (such as `homepage` and `tagline`) have many missing values, while the columns that describe a movie's content (`overview`, `genres`, `keywords`, `cast`, `crew`) are almost complete. That observation drives the order of the next two steps.

3.3 Selecting features

A content-based recommender only needs columns that describe what a movie is about. Seven columns are kept: `id`, `title`, `overview`, `genres`, `keywords`, `cast` and `crew` (from which the director is extracted). Budget, revenue, popularity, release date and similar columns are discarded.

3.4 Cleaning the data

Rows with missing values and duplicate rows are removed, **after** feature selection and only on the selected columns. Cleaning first would have been a mistake: `dropna()` on the full table deletes any movie missing *any* value, including columns we were going to discard

anyway. In practice this removed large numbers of valid movies (Toy Story among them) and made them impossible to search for. Selecting first and cleaning second keeps almost the whole dataset.

3.5 Parsing features and creating movie tags

Each JSON-like column is converted into a plain list of words:

- **Genres and keywords:** all names are extracted.
- **Cast:** only the **top three** billed actors are kept, since minor roles add noise.
- **Director:** the single crew member whose job is "Director".
- **Overview:** split into individual words.

Multi-word names then have their spaces removed ("Christopher Nolan" becomes "ChristopherNolan"). Without this, the vectorizer would treat "Christopher" and "Nolan" as separate words and wrongly link unrelated people who share a first name.

All lists are concatenated, joined into one lowercase string, and stored as a single **tags** column. For example, a movie's tags might look like: `sci-fi thriller christophernolan dream mind leonardodicaprio`. This one column is the complete textual fingerprint of the movie.

3.6 Text vectorization

Machine learning works on numbers, so each tags string is converted into a vector of word counts using `CountVectorizer(max_features=5000, stop_words="english")`:

- `max_features=5000` keeps only the 5,000 most frequent words, which limits memory use and discards extremely rare terms.
- `stop_words="english"` removes filler words such as "the" and "and" that say nothing about a movie.

The result is a matrix with one row per movie and one column per vocabulary word.

3.7 Cosine similarity

Cosine similarity measures the angle between two vectors. Two movies whose tag vectors point in the same direction score close to 1; movies with nothing in common score close to 0. Because it depends on direction rather than length, a movie with a long overview is not unfairly favoured over one with a short overview. The output is a square matrix where entry `[i][j]` is the similarity between movie *i* and movie *j*.

3.8 The recommendation function

The `recommend()` function follows the logic given in the project guideline:

1. Find the row index of the movie the user typed (matching is case-insensitive, with a friendly message if the title is not found).
2. Take that movie's row from the similarity matrix and pair every score with its movie index using `enumerate`.
3. Sort the pairs by score, highest first.
4. Skip the first result (the movie compared with itself, always a perfect match) and return the next five titles.

The finished program asks the user for a movie name in a loop and prints recommendations until they type `quit`.

4. Implementation

The project is delivered in several forms so it can be read, run and submitted easily:

File	Purpose
<code>movie_recommender.ipynb</code>	Jupyter Notebook with a markdown explanation above every step
<code>movie_recommender.py</code>	Modular script version built from functions
<code>movie_recommender_beginner.py</code>	Simplified top-to-bottom script with step-by-step comments
<code>README.md</code>	Setup, usage and deliverables checklist

The core of the implementation matches Modules 10 and 11 of the guideline:

`fit_transform` on the tags column, `cosine_similarity` on the resulting vectors, then `enumerate`, sort and slice `[1:6]` in `recommend()`. The only additions are `max_features`, `stop_words`, case-insensitive matching and a not-found check.

5. Results

The guideline's reference example is the input **Inception**, with the expected recommendations Interstellar, The Prestige, Tenet, The Matrix and Shutter Island. Running the system on Inception is the first test, and the project guideline also suggests testing Titanic, Avatar, Avengers, The Dark Knight and Interstellar.

Record the exact output of your own run in the table below. Results depend on the dataset version and vectorizer settings, so they will not necessarily match the guideline's illustration word for word.

Input movie	Top 5 recommendations
Inception	1. Duplex 2. The Helix... Loaded 3. Star Trek II: The Wrath of Khan 4. Transformers: Revenge of the Fallen 5. Timecop
The Dark Knight	1. The Dark Knight Rises 2. Batman Begins 3. Batman Returns 4. Batman Forever 5. Batman & Robin
Interstellar	1. Silent Running 2. The Martian 3. Space Pirate Captain Harlock 4. Apollo 13 5. Space Cowboys
Titanic	1. Captain Phillips 2. The Notebook 3. Ghost Ship 4. In the Heart of the Sea 5. Poseidon
Avatar	1. Titan A.E. 2. Small Soldiers 3. Ender's Game 4. Independence Day 5. Aliens vs Predator: Requiem

Input movie	Top 5 recommendations
The Avengers	1. Avengers: Age of Ultron 2. Captain America: Civil War 3. Iron Man 3 4. Captain America: The First Avenger 5. Iron Man
Toy Story	1. Toy Story 2 2. Toy Story 3 3. The 40 Year Old Virgin 4. Heartbeeps 5. Max Keeble's Big Move

What a good result looks like. Recommendations should share at least one strong signal with the input: the same director (Nolan films recommending each other), the same franchise or cast (Avengers sequels), or overlapping genres and themes (space travel, heists, romance).

6. Insights and Observations

- 1. Order of preprocessing matters.** Cleaning before selecting features silently deleted valid movies, including Toy Story. Always reduce to the columns you need before dropping missing values.
- 2. Tokenisation details change results.** Removing spaces inside names (so each person or keyword is one token) prevents false matches between people who only share a first name.
- 3. Combining features into one field is simple and effective.** A single `tags` string lets one vectorizer handle plot, genre, people and themes together, with no need for separate models.
- 4. Overview text carries a lot of weight.** An overview contributes dozens of words, while the director contributes one token and the cast three. Plot wording can therefore dominate the similarity score. Giving names and genres more weight would shift recommendations towards "same creators" over "similar plot".
- 5. Franchises and sequels rank highly.** Sequels share cast, keywords and plot vocabulary, so they tend to be top matches. This is usually desirable, but it can crowd out more diverse suggestions.

6. **Cosine similarity handles uneven text length well.** Because it compares direction rather than magnitude, movies with long and short descriptions are compared fairly.
7. **No user data is needed.** A content-based approach works for any movie with metadata, and avoids the cold-start problem faced by collaborative filtering (which needs user ratings).

7. Limitations

- **Static, dated data.** The dataset stops at around 2017, so newer films cannot be recommended or searched.
- **Exact title matching.** The user must type the title as it appears in the dataset (case does not matter, but spelling does). Titles are not searched approximately.
- **No personalisation.** Every user gets the same list for the same movie. The system does not learn individual taste.
- **Memory use.** The dense vector matrix (about 4,800 by 5,000) and the similarity matrix (about 4,800 by 4,800, roughly 185 MB as 64-bit floats) are held in memory. This is fine for 5,000 movies, but would not scale to hundreds of thousands.
- **Simple word counts.** `CountVectorizer` treats every word equally and ignores meaning, so synonyms are not recognised as related.
- **Duplicate titles.** If two different movies share a title, only the first match is used.

8. Future Improvements

- Replace `CountVectorizer` with **TF-IDF**, which down-weights very common words and up-weights distinctive ones.
- **Weight features** (for example repeat genres, director and cast tokens) to control how strongly each influences similarity.
- Add **fuzzy title matching** or suggestions for misspelled titles.
- Include **ratings and popularity** to rank or filter results among similar movies.
- Build a **web interface** (for example Streamlit) with a search box and movie posters, like the interface shown in the guideline.
- Add **collaborative filtering** or a hybrid model once user rating data is available.
- Use **word embeddings** to capture meaning beyond exact word matches.

9. Conclusion

The project delivers a working content-based movie recommender that follows the full guideline pipeline: loading and exploring the dataset, selecting features, cleaning, building combined movie tags, vectorizing with `CountVectorizer`, computing cosine similarity, and generating the top five recommendations for any movie the user enters. Along the way it showed how much small preprocessing decisions, such as the order of cleaning and the handling of multi-word names, affect the final recommendations. The system is simple, fast and explainable, and it provides a solid base for the improvements listed above.